Computer Architecture

Dynamic Scheduling

Special thanks to Prof. Daniel Sorin, Prof. Milo Martin, Prof. Amir Roth, Prof. Mark Hill, and Prof. David Wood. These slides heavily borrow theirs.

Dynamic Scheduling

- Basic pipeline started with single, **in-order issue**, single-cycle operations
- Extended this basic pipeline with
  - Multi-cycle operations
  - Multiple issue (superscalar)

- Now: **dynamic scheduling (out-of-order issue/execution)**
  - **Scoreboard: OoO without solving WAW/WAR**
  - Tomasulo's algorithm: OoO + register renaming to fix WAR/WAW
- **Next half unit:** dynamic scheduling ...
  - Dynamic scheduling + precise state + speculation
  - Advanced topic: dynamic load scheduling

Dynamic Scheduling: Motivation

	1	2	3	4	5	6	7	8	9	10	11
divf f0,f2,f4	F	D	E/	E/	E/	E/	W				
addf f6,f0,f2		F	**D***	**D***	**D***	D	E+	E+	W		
mulf f8,f2,f4			**F***	**F***	**F***	**F**	**D***	D	E*	E*	W

Dynamic Scheduling: Motivation

	1	2	3	4	5	6	7	8	9	10	11
divf f0,f2,f4	F	D	E/	E/	E/	E/	W				
addf f6,f0,f2		F	**D***	**D***	**D***	D	E+	E+	W		
mulf f8,f2,f4			**F***	**F***	**F***	**F**	**D***	D	E*	E*	W

- Cycle 4: **addf** stalls due to ***RAW*** *hazard*
  - OK, fundamental problem
- Also cycle 3: **mulf** stalls due to *pipeline hazard (addf stalls)*
  - Why? **mulf** can't proceed into D because addf is there
  - … but that's the only reason ⇒ not good enough!
- Why can't we decode **mulf** in c4 and execute it in c5?
  - No fundamental reason why we can't do this!

Dynamic Scheduling

- **Dynamic scheduling** (out-of-order execution)
  - Execute instructions in non-sequential (non-von Neumann) order
    + Reduce stalls
    + Improve functional unit utilization
    + Enable parallel execution (not in-order ⇒ can be in parallel)

- Make it appear like sequential execution
  - Extremely important
    – But hard to get correct

Scheduling

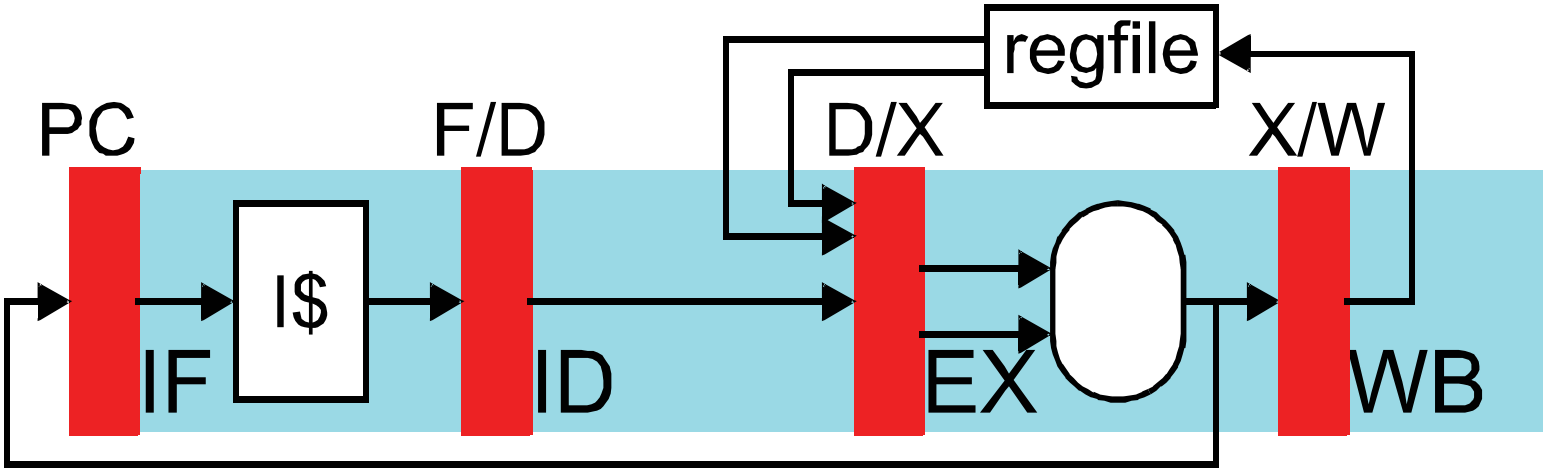
- **Scheduling:** re-arranging instructions to maximize performance
  - Requires knowledge about **structure of processor**
  - Requires knowledge about latencies and dependencies

- Two options for who should schedule instructions
  - **Dynamic scheduling:** by hardware (today; our focus)
  - Static scheduling: by compiler

Before We Proceed …

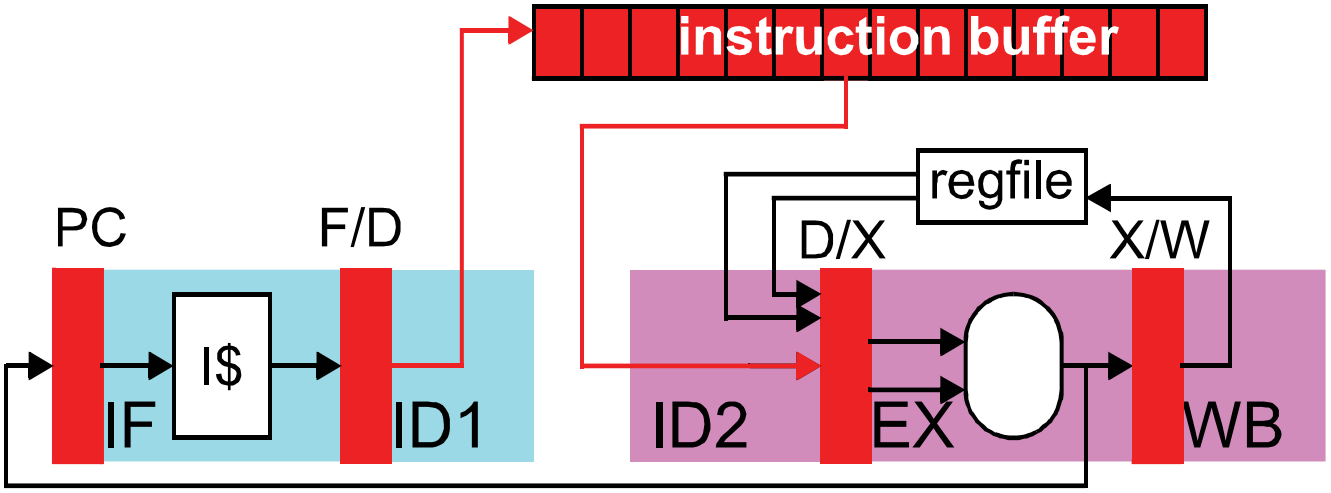
- Why build complicated hardware if we can do this in software?
  - + Performance portability
    - Don't want to recompile for new machines
  - + More information available to hardware
    - Addresses, branch directions, cache misses unknown to compiler
  - + More resources available to hardware
    - May not have enough architectural registers to fix WAR/WAW
  - + Easier to speculate in hardware
    - Easier to recover from miss-speculation
  - – But compiler can look at more instructions
    - It's possible to do a combination of both
    - Compiler does as much as it can, hardware does rest

The Problem with In-Order Pipelines



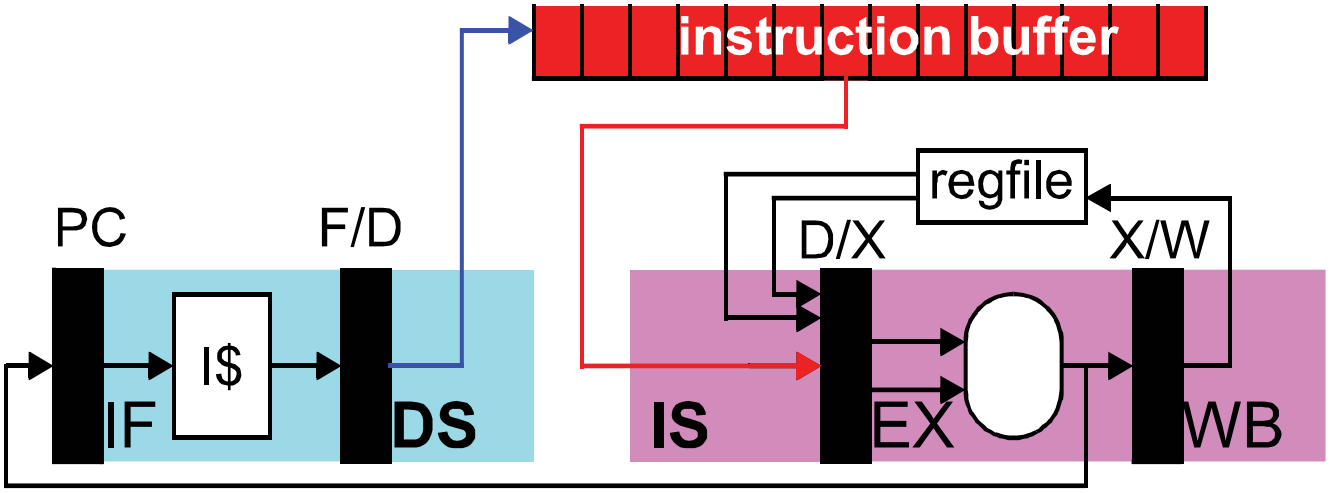
- In-order pipeline
  - Simple 4-stage: IF, ID, EX (multiple cycle, includes M), WB
  - **Structural hazard**: 1 pipeline register per stage
    - 1 instruction per stage per cycle (unless pipe is replicated)
    - Younger instruction can't pass older without "killing" it
- Out-of-order pipeline
  - Implements "passing" functionality

Instruction Buffer



- Trick: **instruction buffer** (many names for this buffer)
  - Basically: a bunch of latches for holding instructions
  - This is the scope of instructions that the scheduler can see
- Split ID into two pieces
  - Accumulate decoded instructions in buffer **in-order**
  - Buffer sends instructions down rest of pipe **out-of-order**

Dispatch and Issue



- **Dispatch (DS):** first part of ID
  - Allocate resources in instruction buffer
    - New kind of structural hazard (instruction buffer could be full)
  - Dispatch is in-order, and stall propagates to younger instructions
- **Issue (IS):** second part of ID
  - Send instructions from instruction buffer to execution units
  - Out-of-order, stalls do NOT propagate to younger instructions

Dispatch Method #1: Scoreboarding

- Instruction buffer ⇒ **scoreboard**
  - **Centralized control scheme**
    - No bypassing
    - No elimination of WAR/WAW hazards

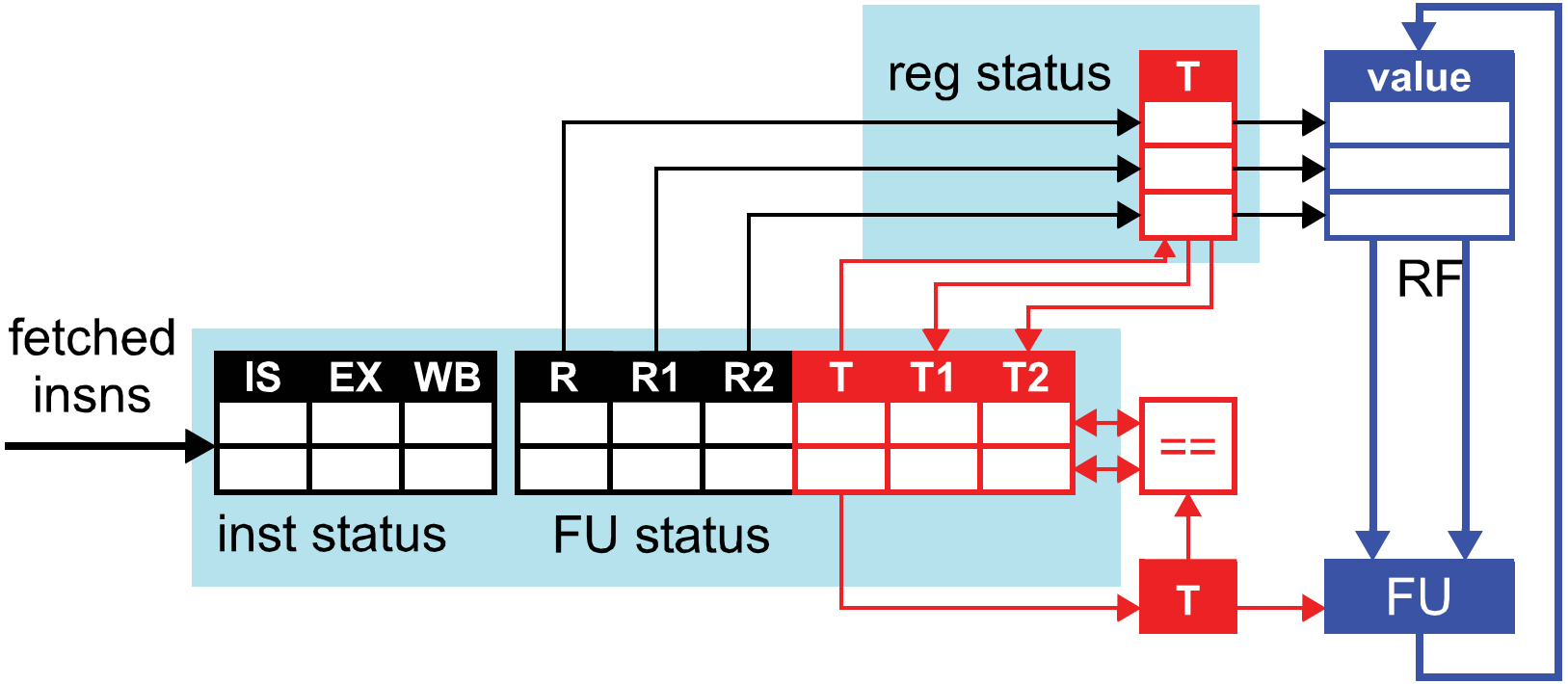
- First implementation: CDC6600 [1964]
  - 16 separate non-pipelined functional units
  - 4 FP, 5 memory, 7 integer

- Our example: Simple Scoreboard
  - 5 functional units: 1 ALU, 1 load, 1 store, 2 FP (3-cycle, pipelined)
    - All units take 1 cycle except FP
  - For simplicity, assume 1-wide pipeline (not superscalar, but applies to larger widths)

Scoreboard Data Structures

- **Instruction status**: 1 entry per "active" instruction
  - Which stage instruction is in (presence in scoreboard implies DS)
- **Functional unit (FU) status**: 1 entry per FU
  - **busy:** FU is busy, **op:** current operation
  - **R1,R2, R:** source and destination registers
  - **T1, T2:** tags of FUs producing source registers
  - **T:** tag of FU producing destination register
- **Register status**: 1 entry per architectural register
  - **T:** tag of FU (if any) that will write the register
- Tag fields interpreted as "ready bits"
  - tag == 0: register value **is** ready (in register file)
  - tag != 0: register value is **not** ready (in register file)

Simple Scoreboard

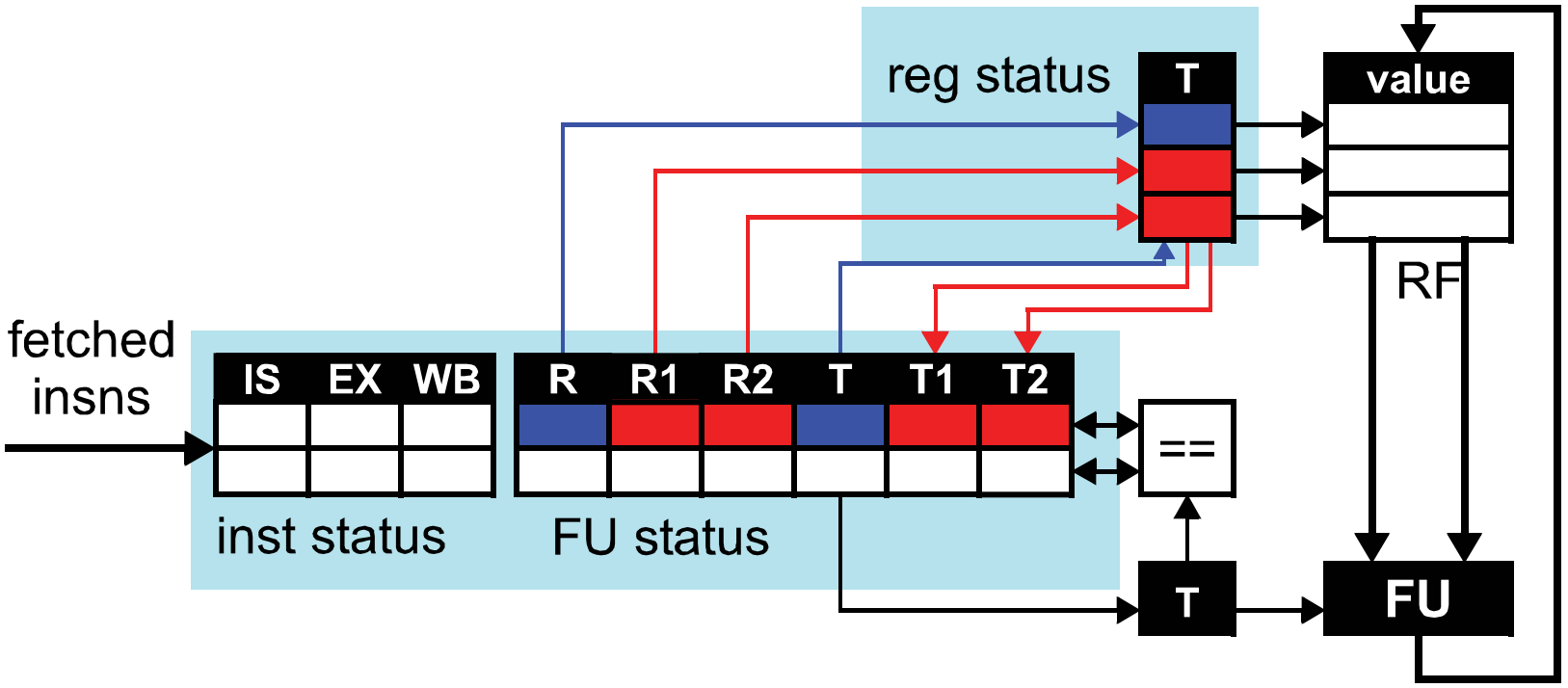


- Instruction fields and status bits
- Tags
- Values

New Pipeline with Scoreboard

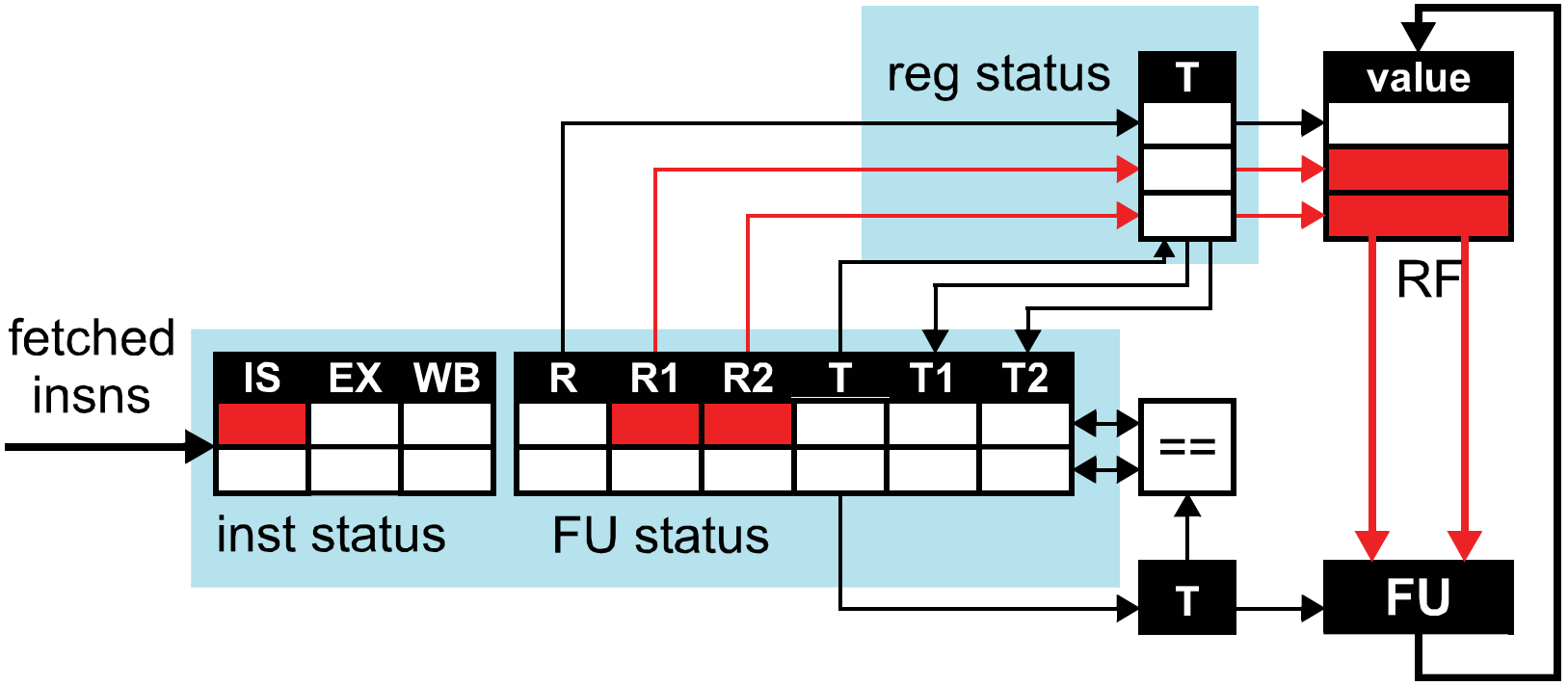
- New pipeline structure: IF, ***DS, IS, EX, WB***
  - **DS (dispatch)** from fetch to the scoreboard
    - (structural hazard/WAW) ? (stall) : (allocate)
  - **IS (issue)** to the functional units
    - (RAW hazard) ? (wait) : (read registers, go directly to execute)
  - **EX (execute)**
    - execute operation, notify scoreboard when done
  - **WB (writeback)**
    - (WAR hazard) ? (wait) : (write register, free scoreboard entry)
- Assume
  - WB and RAW-dependent IS can take place in same cycle
  - WB and structural-dependent DS can take place in same cycle

Scoreboard: Dispatch (DS)



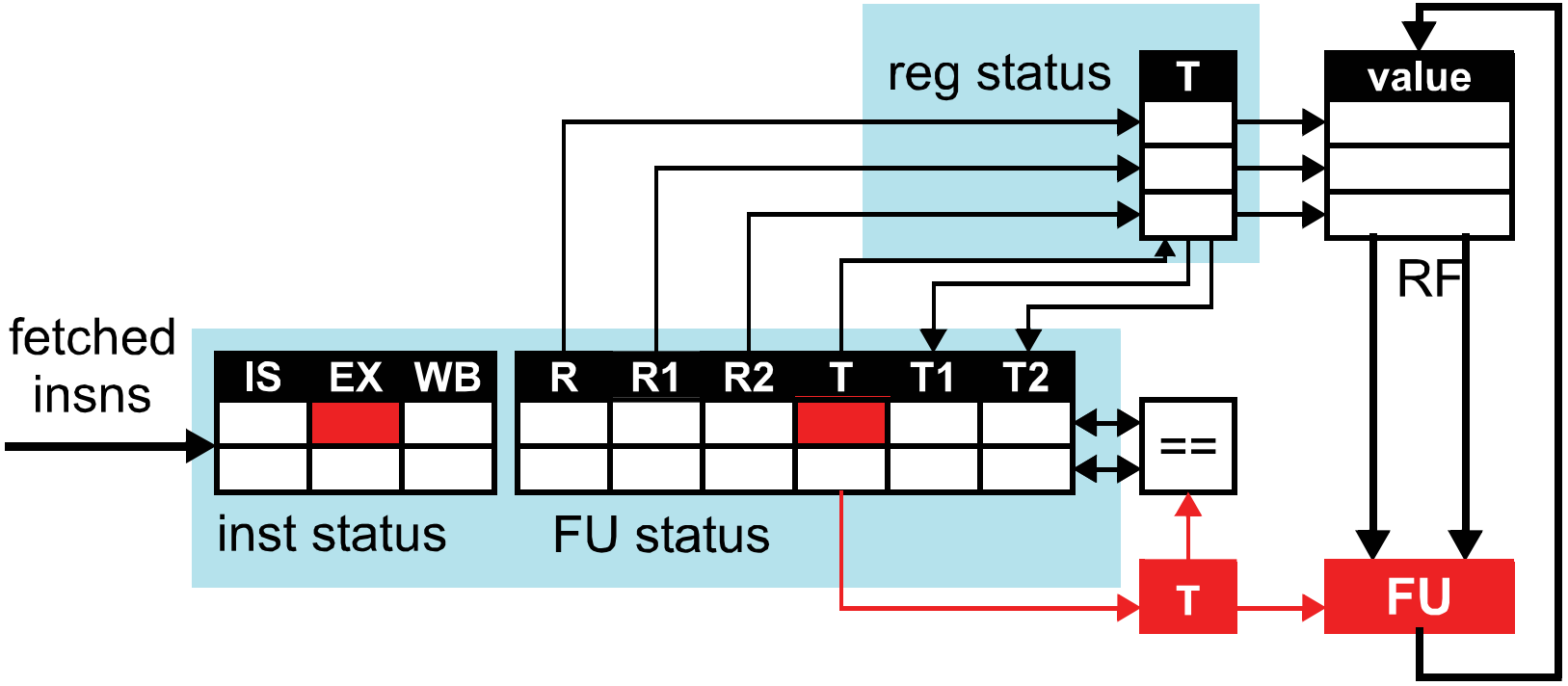
- Stall for WAW and structural hazards, but otherwise:
  - allocate scoreboard entry
  - copy status for input registers
  - set status for output registers

Scoreboard: Issue (IS)

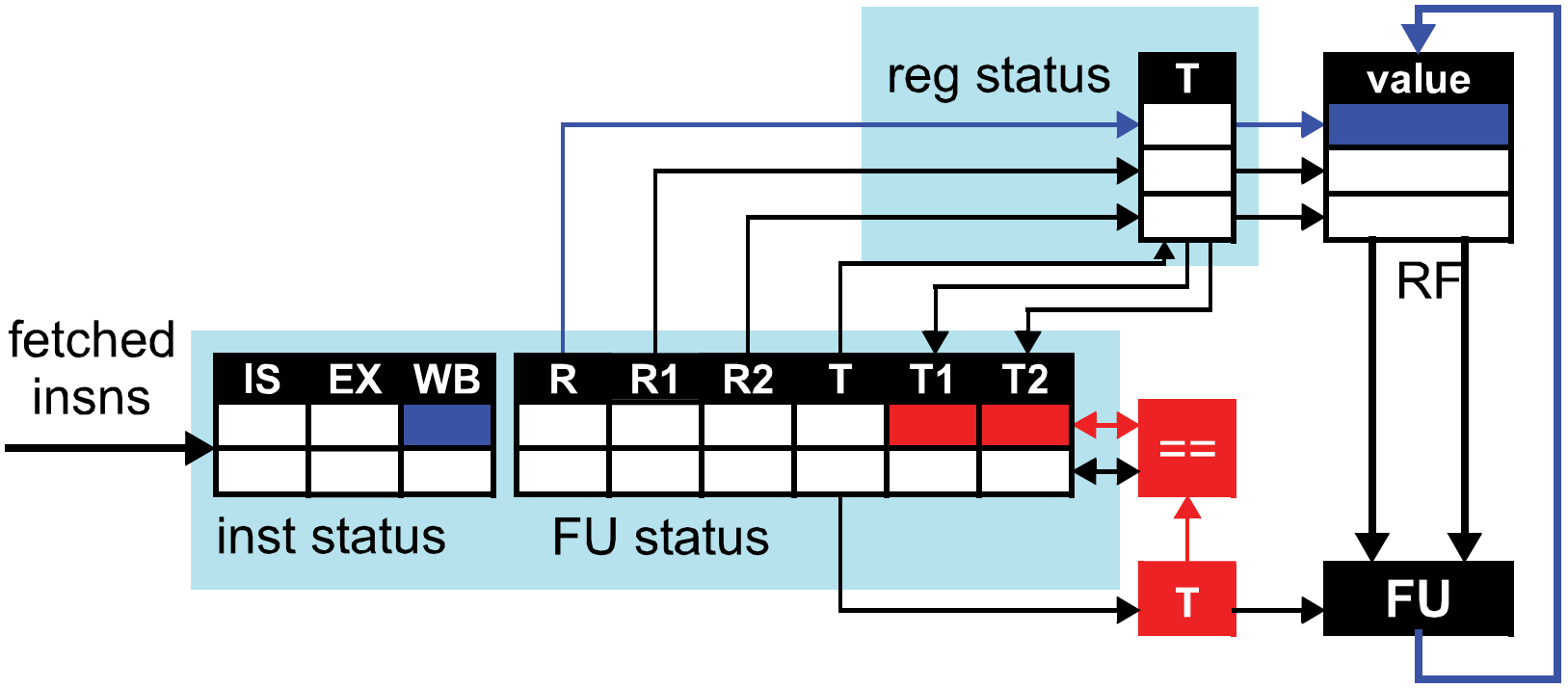


- Wait for RAW hazards (T1 or T2 not empty), but otherwise:
  - read registers

Scoreboard: Execute (EX)



Scoreboard: Writeback (WB)



- Wait for WAR hazards, but otherwise:
  - writeback result
  - compare tags with waiting instructions
  - on match: clear tag (set input to "ready")

Example: SAX

- SAX (Scalar Alpha X): simplified SAXPY

```
DO I = 1,N
        Z[I] = A*X[I]
```

- assembly code:

```
loop:
        ldf f0,X(x1)        // f0=X[i], assume I in x1
        mulf f4,f0,f2       // assume A in f2
        stf f4,Z(x1)        // Z[i]=A*X[i]
        addi x1,x1,#4       // I=I+4
        bge x2,x1,loop      // (x2>=x1)? assume 4N in x2
```

- Consider two iterations by unrolling loop, ignore branch

SB Data Structures

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				
addi x1,x1,#4				
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	
f2	
f4	
x1	

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	No						
load	No						
store	No						
FP1	No						
FP2	No						

SB Example: Cycle 1

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**			
mulf f4,f0,f2				
stf f4,Z(x1)				
addi x1,x1,#4				
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	**load**
f2	
f4	
x1	

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	No						
load	Yes	**ldf**	**f0**		**x1**		
store	No						
FP1	No						
FP2	No						

← Allocate

SB Example: Cycle 2

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**		
mulf f4,f0,f2	**2**			
stf f4,Z(x1)				
addi x1,x1,#4				
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	**load**
f2	
f4	**FP1**
x1	

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	No						
load	Yes	**ldf**	**f0**		**x1**		
store	No						
FP1	Yes	**mulf**	**f4**	**f0**	**f2**	**load**	
FP2	No						

← Allocate

SB Example: Cycle 3

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**	**3**	
mulf f4,f0,f2	**2**			
stf f4,Z(x1)	**3**			
addi x1,x1,#4				
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	**load**
f2	
f4	**FP1**
x1	

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	No						
load	Yes	**ldf**	**f0**		**x1**		
store	Yes	**stf**		**f4**	**x1**	**FP1**	
FP1	Yes	**mulf**	**f4**	**f0**	**f2**	**load**	
FP2	No						

← Allocate

←Stalled on RAW

SB Example: Cycle 4

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**	**3**	**4**
mulf f4,f0,f2	**2**	**4**		
stf f4,Z(x1)	**3**			
addi x1,x1,#4	**4**			
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	~~**load**~~
f2	
f4	**FP1**
x1	**ALU**

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	Yes	**addi**	**x1**	**x1**			
load	No						
store	Yes	**stf**		**f4**	**x1**	**FP1**	
FP1	Yes	**mulf**	**f4**	**f0**	**f2**	~~**load**~~	
FP2	No						

← **Allocate**

← **Free**

← **Stalled on RAW (f4)**

f0 is now ready

SB Example: Cycle 5

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**	**3**	**4**
mulf f4,f0,f2	**2**	**4**	**5**	
stf f4,Z(x1)	**3**			
addi x1,x1,#4	**4**	**5**		
ldf f0,X(x1)	**5**			
mulf f4,f0,f2				
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	**load**
f2	
f4	**FP1**
x1	**ALU**

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	Yes	**addi**	**x1**	**x1**			
load	Yes	**ldf**	**f0**		**x1**		**ALU**
store	Yes	**stf**		**f4**	**x1**	**FP1**	
FP1	Yes	**mulf**	**f4**	**f0**	**f2**		
FP2	No						

← Allocate

← Stalled on RAW (f4)

SB Example: Cycle 6

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**	**3**	**4**
mulf f4,f0,f2	**2**	**4**	**5+**	
stf f4,Z(x1)	**3**			
addi x1,x1,#4	**4**	**5**	**6**	
ldf f0,X(x1)	**5**			
mulf f4,f0,f2				
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	**load**
f2	
f4	**FP1**
x1	**ALU**

← DS Stalls: WAW hazard w/ mulf (f4)

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	Yes	**addi**	**x1**	**x1**			
load	Yes	**ldf**	**f0**		**x1**		**ALU**
store	Yes	**stf**		**f4**	**x1**	**FP1**	
FP1	Yes	**mulf**	**f4**	**f0**	**f2**		
FP2	No						

SB Example: Cycle 7

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**	**3**	**4**
mulf f4,f0,f2	**2**	**4**	**5+**	
stf f4,Z(x1)	**3**			
addi x1,x1,#4	**4**	**5**	**6**	
ldf f0,X(x1)	**5**			
mulf f4,f0,f2				
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	**load**
f2	
f4	**FP1**
x1	**ALU**

WB Stall: WAR hazard w/ stf (x1)

DS Stall: WAW hazard w/ mulf (f4)

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	Yes	**addi**	**x1**	**x1**			
load	Yes	**ldf**	**f0**		**x1**		**ALU**
store	Yes	**stf**		**f4**	**x1**	**FP1**	
FP1	Yes	**mulf**	**f4**	**f0**	**f2**		
FP2	No						

SB Example: Cycle 8

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**	**3**	**4**
mulf f4,f0,f2	**2**	**4**	**5+**	**8**
stf f4,Z(x1)	**3**	**8**		
addi x1,x1,#4	**4**	**5**	**6**	
ldf f0,X(x1)	**5**			
mulf f4,f0,f2	**8**			
stf f4,Z(x1)				

Register Status	
Register	**T**
f0	**load**
f2	
f4	~~**FP1**~~ **FP2**
x1	**ALU**

First mulf (FP1) is finished

WB Stall: WAR hazard w/ stf (x1)

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	Yes	**addi**	**x1**	**x1**			
load	Yes	**ldf**	**f0**		**x1**		**ALU**
store	Yes	**stf**		**f4**	**x1**	~~**FP1**~~	
FP1	No						
FP2	Yes	**mulf**	**f4**	**f0**	**f2**	**load**	

f4 is ready

← Free

← Allocate

SB Example: Cycle 9

Instruction Status				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5+	8
stf f4,Z(x1)	3	8	9	
addi x1,x1,#4	4	5	6	9
ldf f0,X(x1)	5	9		
mulf f4,f0,f2	8			
stf f4,Z(x1)				

Register Status	
Register	T
f0	load
f2	
f4	FP2
x1	~~ALU~~

addi written

←DS Stall: structural hazard (store unit busy)

Functional Unit Status							
T	Busy	op	R	R1	R2	T1	T2
ALU	No						
load	Yes	ldf	f0		x1		~~ALU~~
store	Yes	stf		f4	x1		
FP1	No						
FP2	Yes	mulf	f4	f0	f2	load	

← Free

← x1 is ready

SB Example: Cycle 10

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**	**3**	**4**
mulf f4,f0,f2	**2**	**4**	**5+**	**8**
stf f4,Z(x1)	**3**	**8**	**9**	**10**
addi x1,x1,#4	**4**	**5**	**6**	**9**
ldf f0,X(x1)	**5**	**9**	**10**	
mulf f4,f0,f2	**8**			
stf f4,Z(x1)	**10**			

Register Status	
Register	**T**
f0	**load**
f2	
f4	**FP2**
x1	

← **WB and dependent DS in same cycle**

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	No						
load	Yes	**ldf**	**f0**		**x1**		
store	Yes	**stf**		**f4**	**x1**	**FP2**	
FP1	No						
FP2	Yes	**mulf**	**f4**	**f0**	**f2**	**load**	

← **Free then allocate**

SB Example: Cycle X …

Instruction Status				
Instruction	**DS**	**IS**	**EX**	**WB**
ldf f0,X(x1)	**1**	**2**	**3**	**4**
mulf f4,f0,f2	**2**	**4**	**5+**	**8**
stf f4,Z(x1)	**3**	**8**	**9**	**10**
addi x1,x1,#4	**4**	**5**	**6**	**9**
ldf f0,X(x1)	**5**	**9**	**10**	
mulf f4,f0,f2	**8**			
stf f4,Z(x1)	**10**			

Register Status	
Register	**T**
f0	**load**
f2	
f4	**FP2**
x1	

← **WB and dependent DS in same cycle**

Functional Unit Status							
T	**Busy**	**op**	**R**	**R1**	**R2**	**T1**	**T2**
ALU	No						
load	Yes	**ldf**	**f0**		**x1**		
store	Yes	**stf**		**f4**	**x1**	**FP2**	
FP1	No						
FP2	Yes	**mulf**	**f4**	**f0**	**f2**	**load**	

Notice, OoOness!

← **Free then allocate**

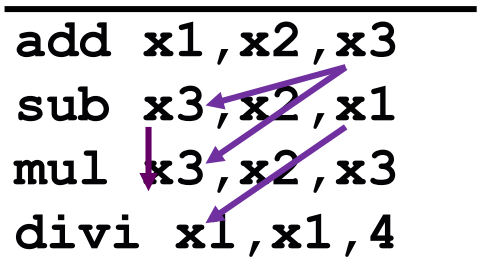
SB Redux

+ Cheap hardware
  - Scoreboard is cheap
- Pretty good performance
  - 1.7X for FORTRAN programs
  - 2.5X for hand-coded assembly
– No bypassing
  - RAW dependencies handled through registers
– Limited scheduling scope
  - WAW/structural hazards force in-order dispatch
  - WAR hazards delay write-back and issue of dependent operations

Data Dependencies

- RAW dependencies are fundamental
  - We have to wait until they are resolved

- Is it true for WAR and WAW?
  - **WAR**
    - x1 of sub and x1 of divi
    - x3 of sub and x3 of add
    - x3 of mul and x3 of add
  - **WAW**
    - x3 of sub and x3 of mul
    - x3 of divi and x3 of add



- Can we execute divi and sub in parallel?
  - Requires removing WAR/WAW dependencies

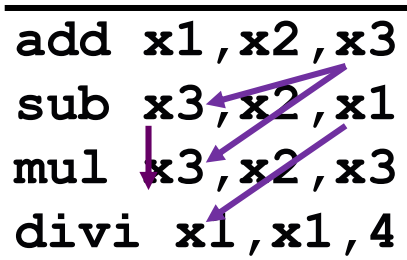
Eliminating WAR and WAW

- Can we execute divi in parallel with sub?
  - Register renaming

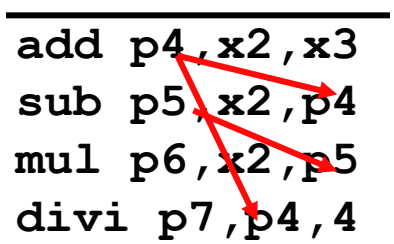
- Approach
  - (step #1) **rename registers (2 methods)**
  - (step #2) schedule

- Example

```
add x1,x2,x3
sub x3,x2,x1
mul x3,x2,x3
divi x1,x1,4
```



```
add p4,x2,x3
sub p5,x2,p4
mul p6,x2,p5
divi p7,p4,4
```



Register Renaming in HW

- **Register renaming** (in hardware)
  - Change register names to eliminate **WAR/WAW** hazards
  - One of the most elegant (yet simple) concepts in computer architecture

- Key: think of architectural registers as names, not locations
  - Can have more locations than names
  - Dynamically map names to locations
  - Map table holds the current mappings (name→location)
    - write: allocate new location and record it in map table
    - read: find location of most recent write by name lookup in map table
    - minor detail: must de-allocate locations appropriately on writeback

Step #1: Register Renaming Example

- Names: **x1,x2,x3** (architected)
- Locations: **p1,p2,p3,p4,p5,p6,p7 (physical)**
- Original mapping: **x1→p1, x2→p2, x3→p3, p4–p7** are "available"

MapTable

x1	x2	x3
p1	p2	p3
p4	p2	p3
p4	p2	p5
p4	p2	p6

FreeList

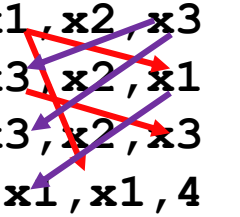
p4,p5,p6,p7
p5,p6,p7
p6,p7
p7

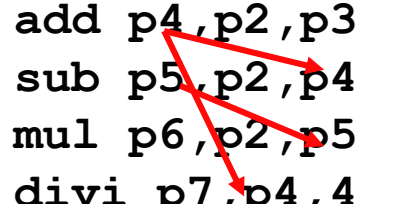
Original insns

```
add x1,x2,x3
sub x3,x2,x1
mul x3,x2,x3
divi x1,x1,4
```

Renamed insns

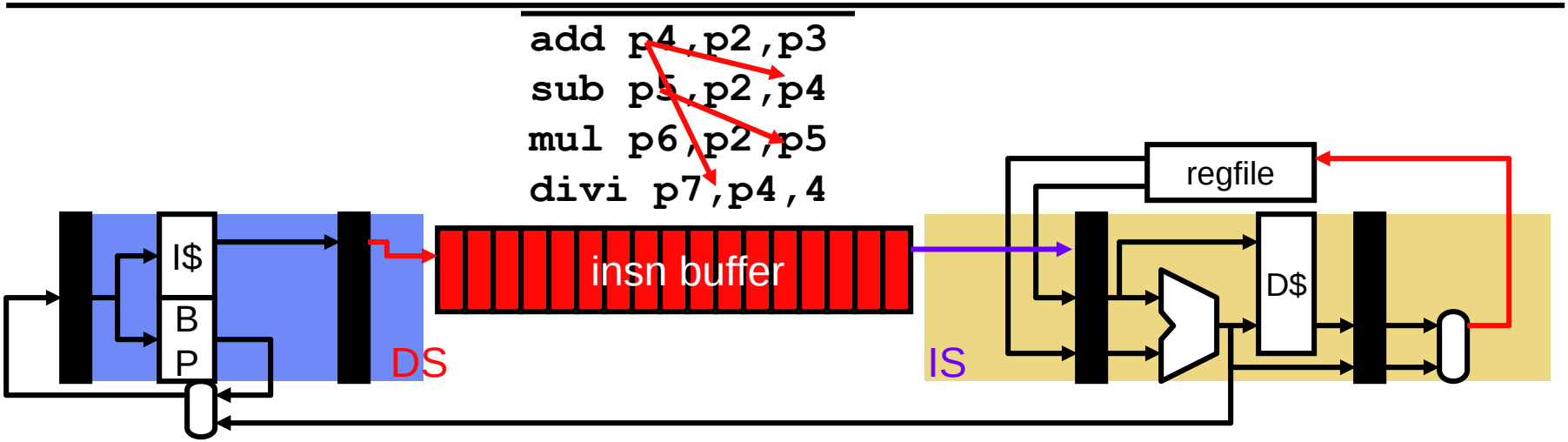
```
add p4,p2,p3
sub p5,p2,p4
mul p6,p2,p5
divi p7,p4,4
```





- Renaming – conceptually write each register once
  - + WAW/WAR
  - + Leaves RAW intact!
- When to reuse a physical register? After insn done

Step #2: Dynamic Scheduling




```
add p4,p2,p3
sub p5,p2,p4  and divi p7,p4,4
mul p6,p2,p5
```

- Notice we can schedule **`sub`** and **`divi`** together

DS Method #2: Tomasulo's Algorithm

- Instruction buffer ⇒ **reservation stations (RS)**
  - Distributed control scheme (Scoreboard was centralized)
    - Uses data bypassing
    - **Common data bus (CDB)** broadcasts results to RS
    - **Register renaming** eliminates WAR/WAW hazards

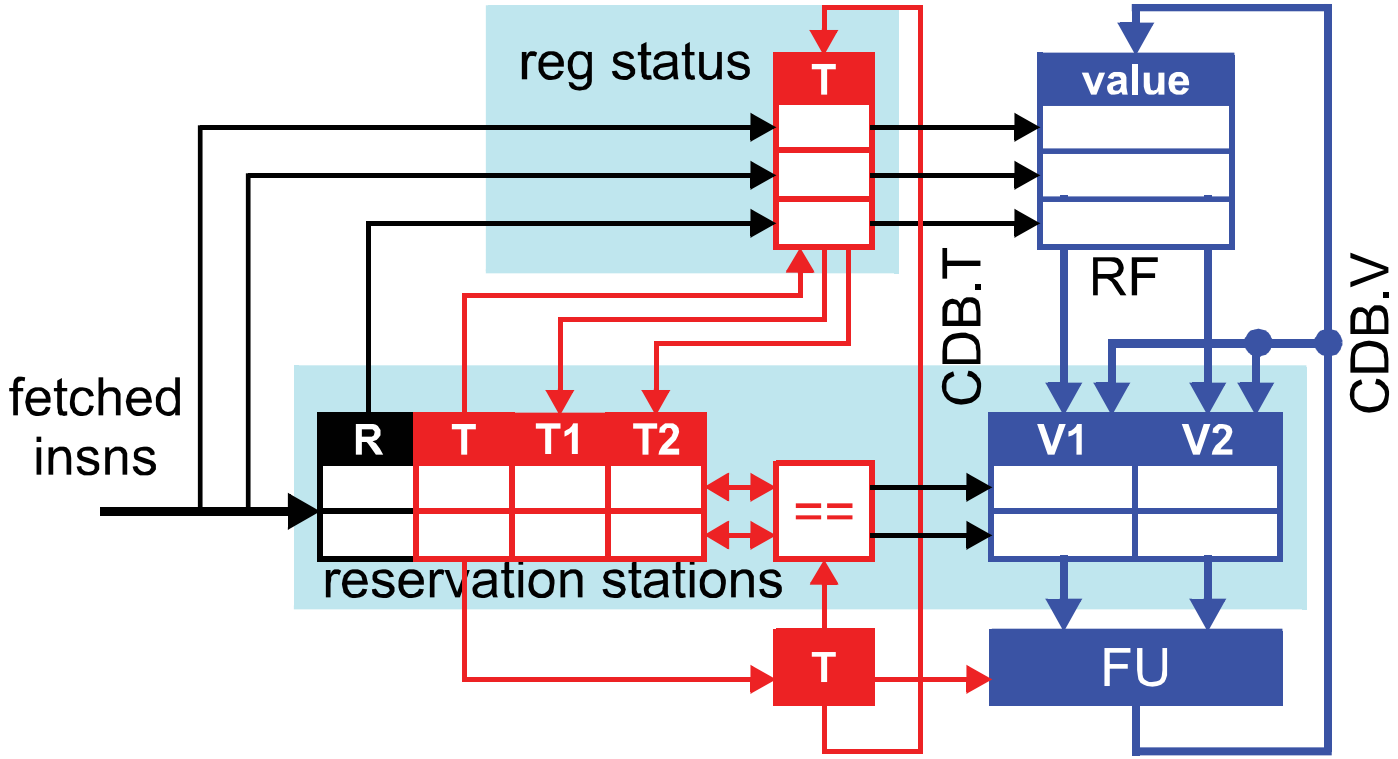
- First implementation: IBM 360/91 [1967]
  - Dynamic scheduling for **FP units only**

- Our example: Simple Tomasulo
  - Dynamic scheduling all insns
  - 5 RS: 1 ALU, 1 load, 1 store, 2 FP (3 cycle, pipelined)
    - All insns except FP taken 1 cycle.

Tomasulo Data Structures

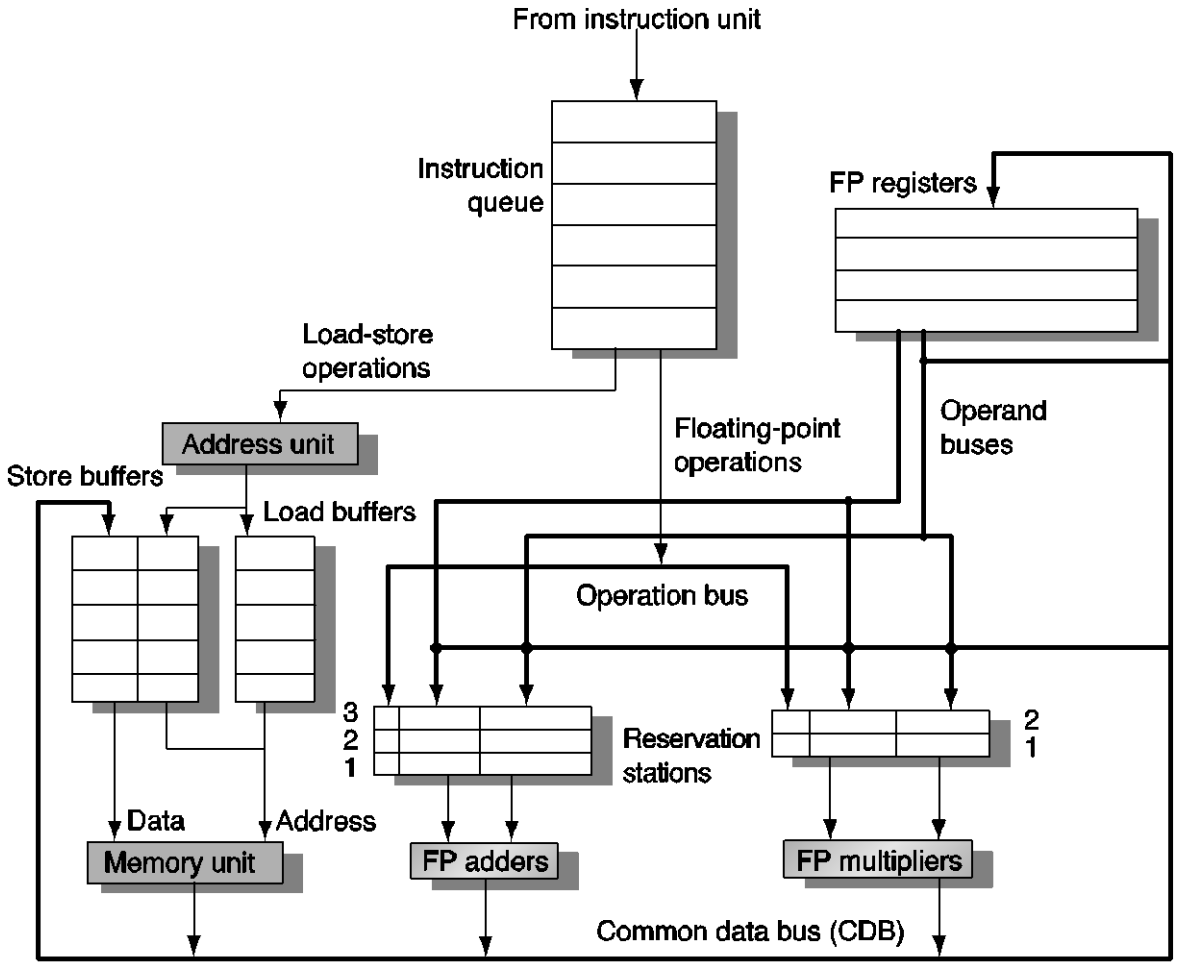
- Reservation stations
  - **busy, FU, op, R**: destination register name
  - **T1,T2**: source register tag (RS# that will produce the value)
  - **T**: destination register tag (RS# of this RS)
  - **V1,V2**: source register value
- Register status
  - **T:** tag (RS# that will write register)
- CDB: common data bus
  - broadcasts <value, tag> of completed instructions
- Tags interpreted as (more sophisticated) ready bits
  - tag == 0? value is ready (somewhere)
  - tag != 0? value is not ready, wait until CDB broadcasts this tag

Simple Tomasulo



- Instruction fields and status bits
- **Tags**
- **Values**

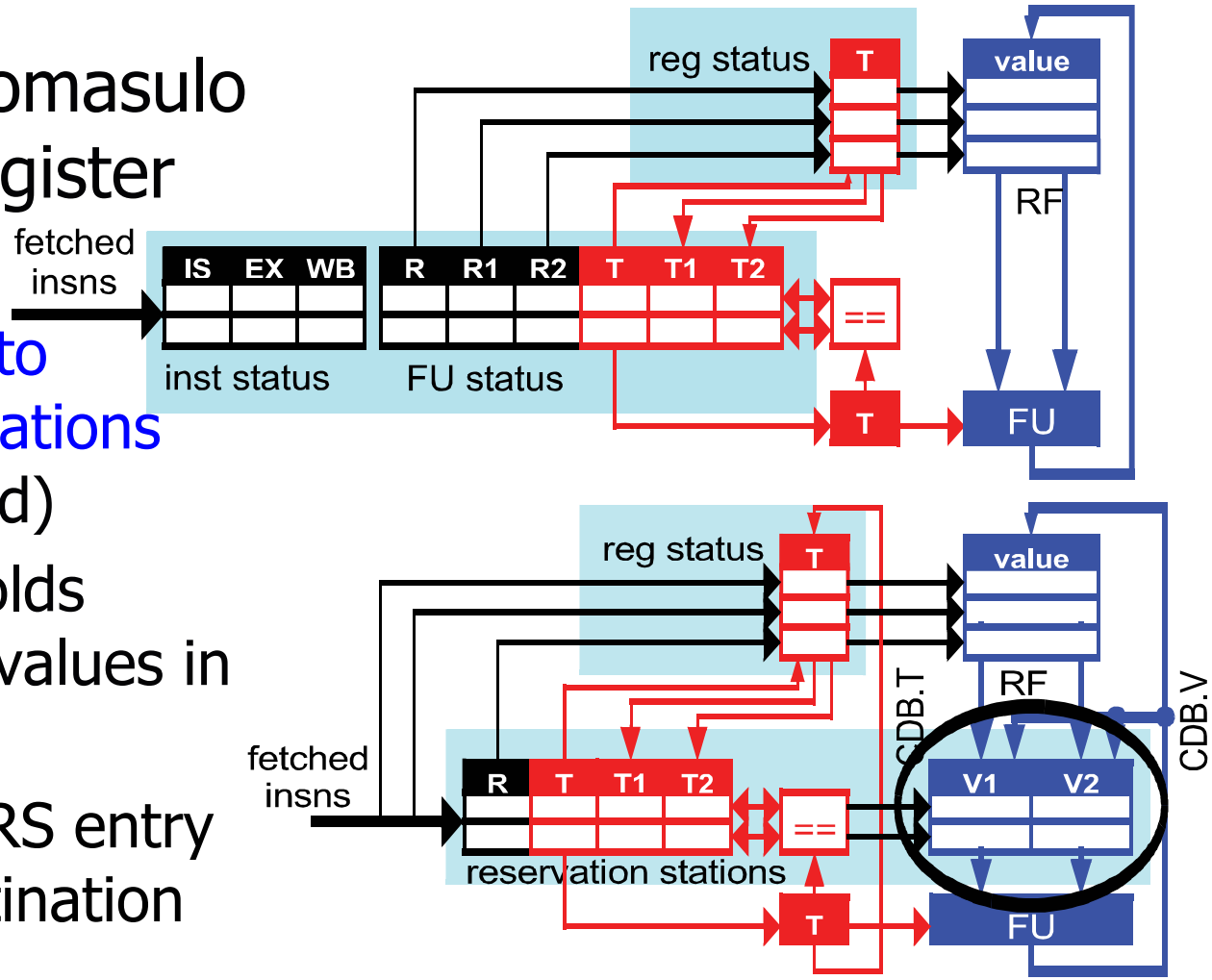
Real Tomasulo



© 2003 Elsevier Science (USA). All rights reserved.

Scoreboard vs. Tomasulo

- What about Tomasulo implements register renaming?
  - Value copied to reservation stations (RS) (renamed)
  - Instruction holds correct input values in its **own** RS
  - Using a new RS entry for same destination registers

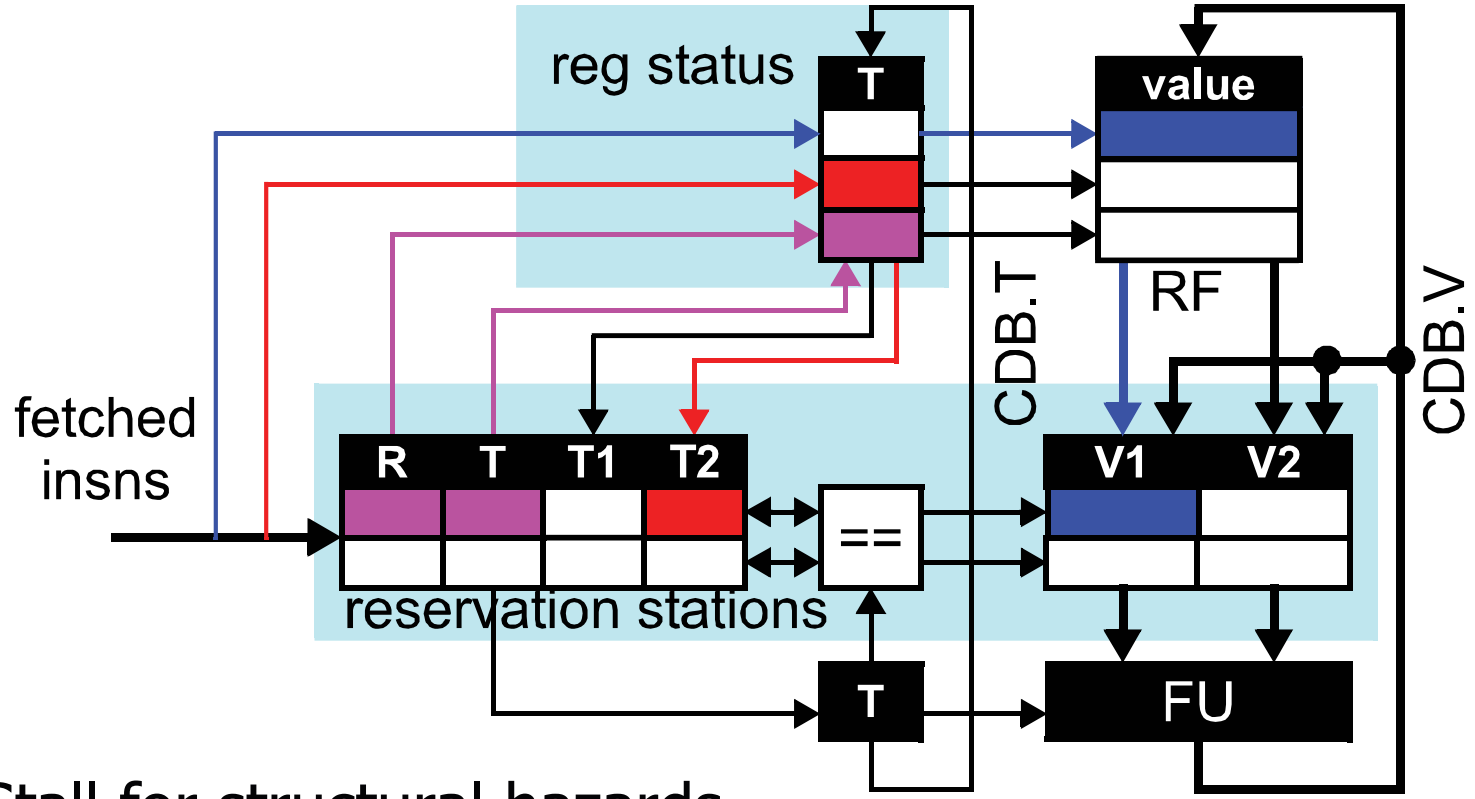


Tomasulo Pipeline

- Pipeline structure: IF, **DS, IS**, **EX**, **WB**
  - **DS (dispatch)**
    - (available RS) ?

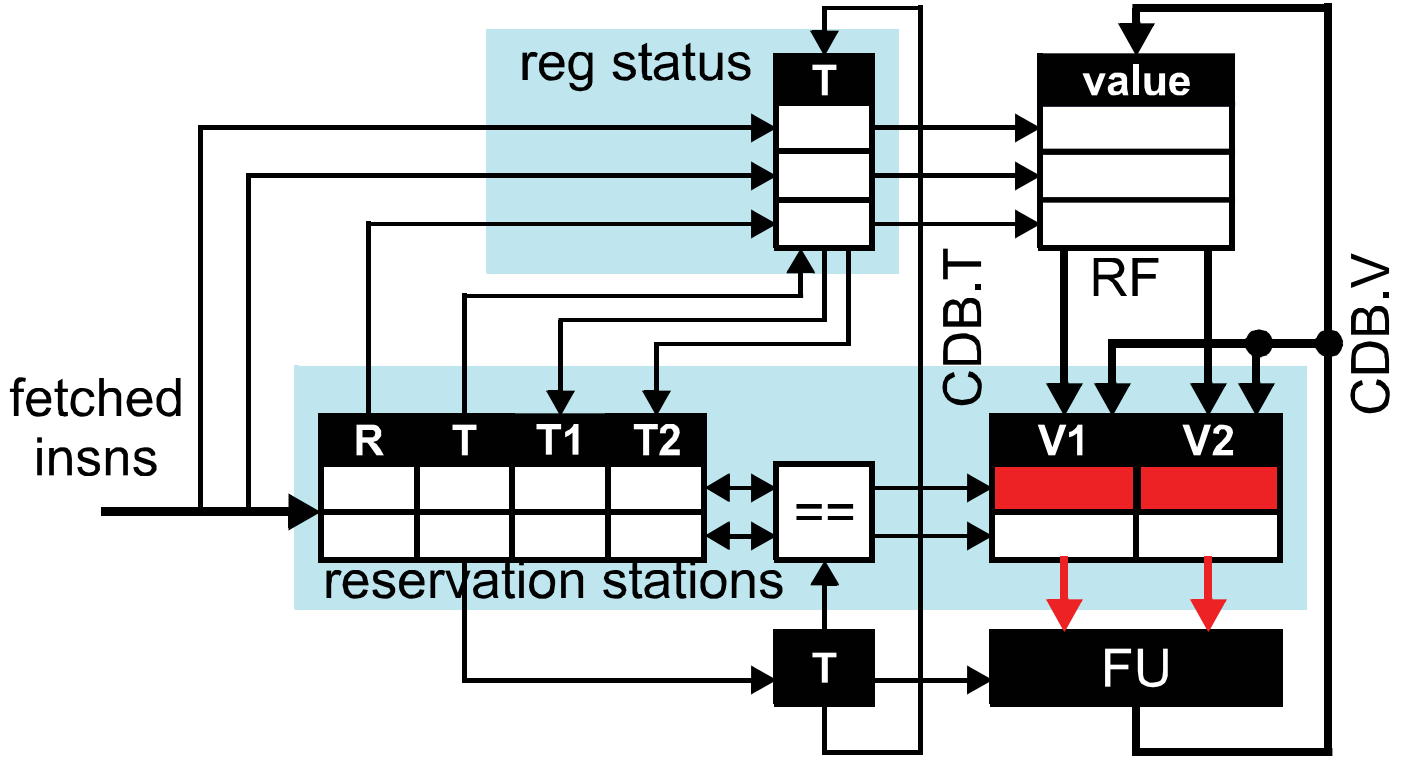
      (allocate RS, **copy ready values**, non-ready tags to RS) : (stall)
  - **IS (issue)**
    - (Wait for RAW -- operands ready) ? (execute) : (wait, monitor CDB)
  - **EX (execute)**
    - As the name suggests …
  - **WB (writeback)**
    - (CDB available) ? (broadcast result, write regf, free RS) : (wait)
  - Assume
    - WB and RAW-dependent IS can go in same cycle
    - WB and structural dependent DS can go in same cycle

Tomasulo: Dispatch (DS)



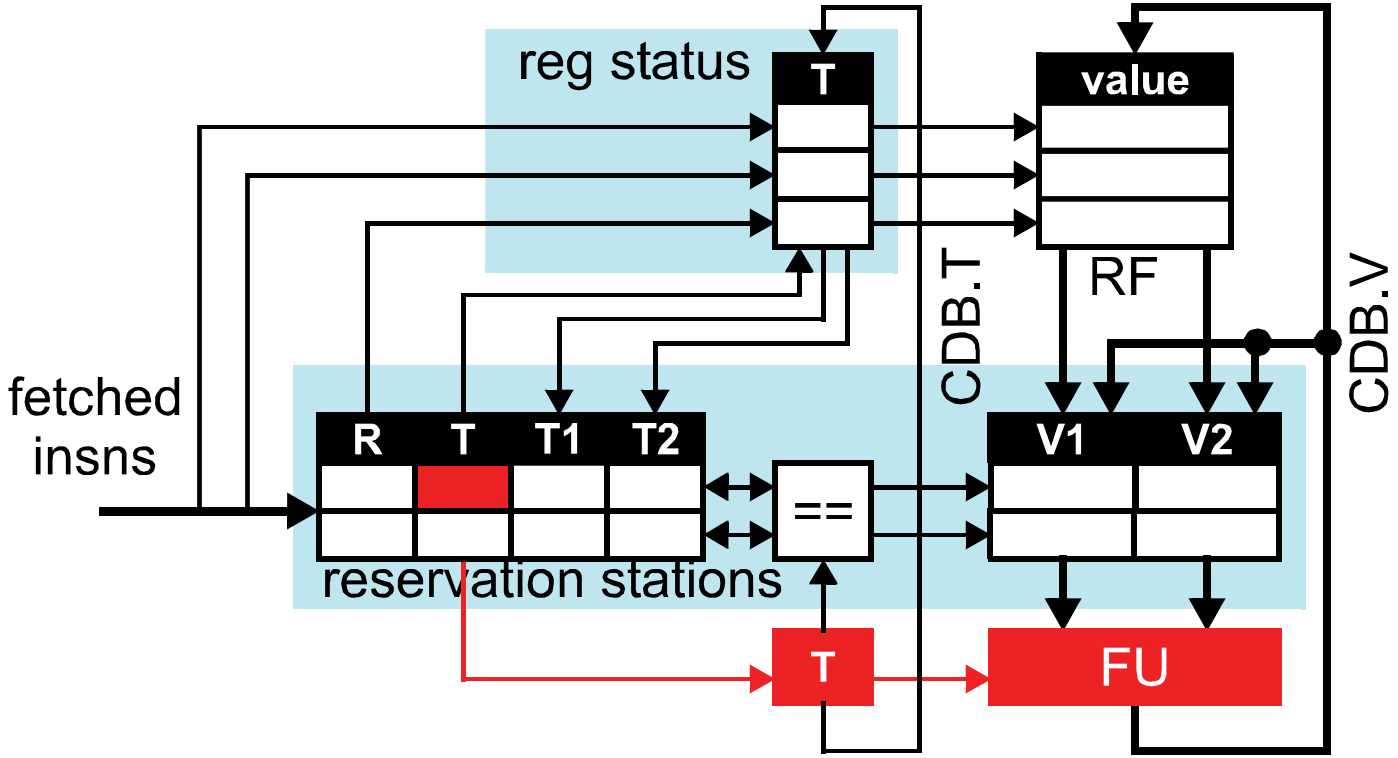
- Stall for structural hazards
  - Read ready register values into RS
  - Read tags for non-ready register values into RS
  - Rename (set status for) new result

Tomasulo: Issue (IS)

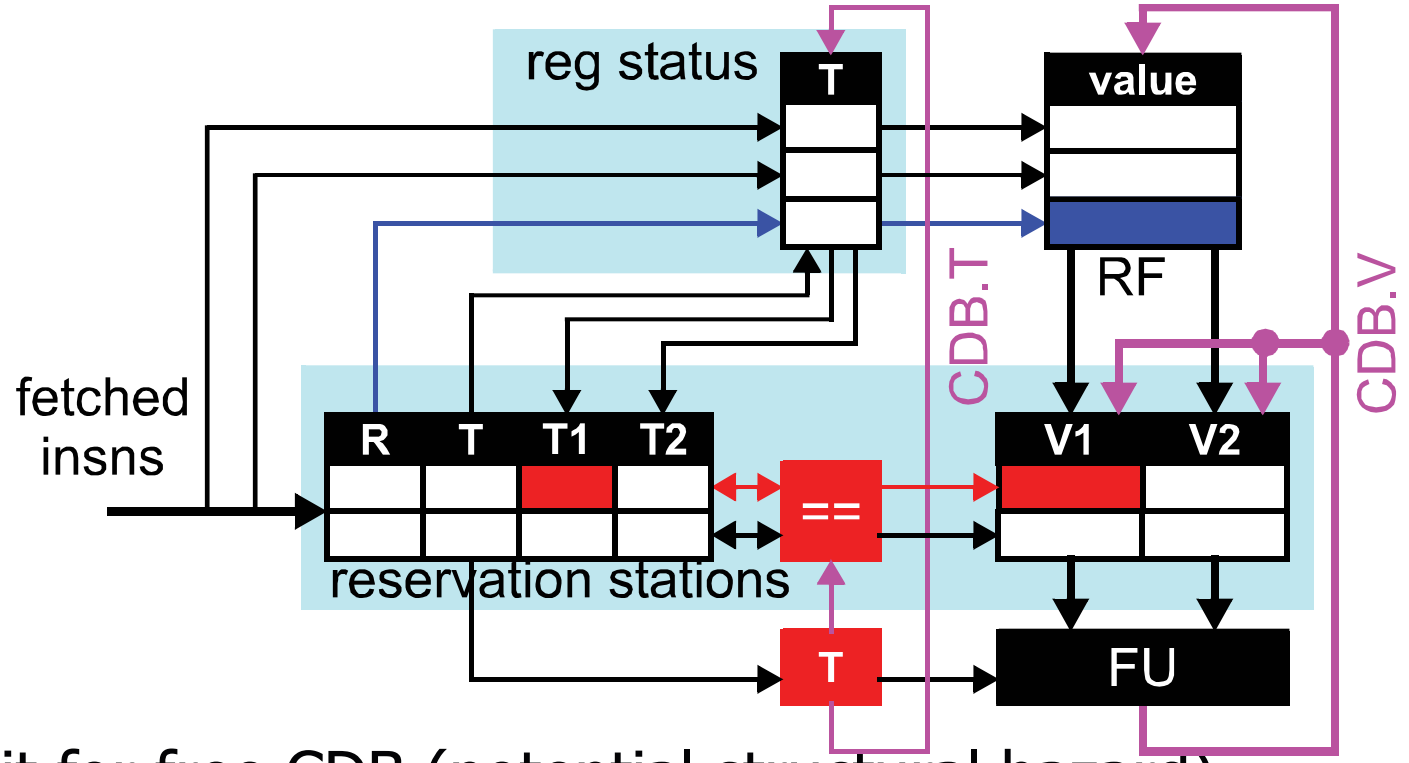


- Wait for RAW hazards to be resolved
  - Read register values from CDB

Tomasulo: Execute (EX)



Tomasulo: Writeback (WB)



- Wait for free CDB (potential structural hazard)
  - Broadcast result (value+tag) on CDB
  - Write result to register file + clear reg status (if tag matches)
  - Compare with RS input tags (match? clear tag + copy value)

Tomasulo Data Structures

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				
addi x1,x1,#4				
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	
f2	
f4	
x1	

CDB	
Reg.	T
x1	

Reservation Stations and Load/Store Buffers									
T	FU	Busy	op	R	R1	R2	T1	T2	addr
1	ALU	No							
2	load	No							
3	store	No							
4	FP1	No							
5	FP2	No							

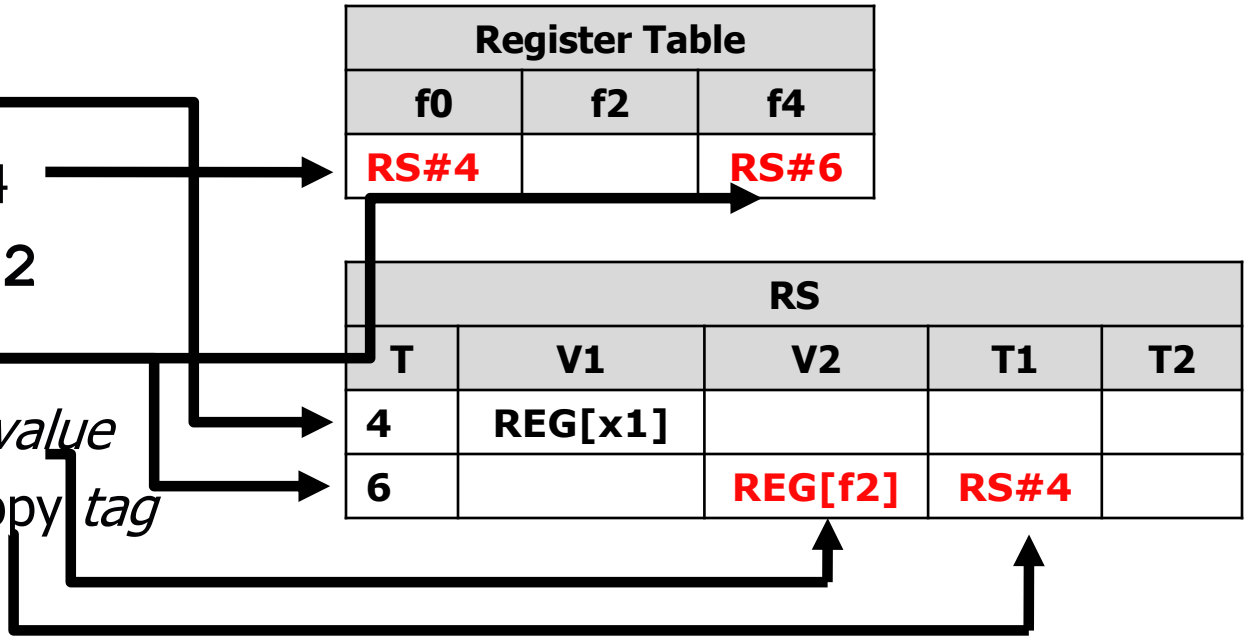
Tomasulo-Style Register Renaming

- **Names:** architectural registers
- **Locations:** registers in register file AND reservation stations (RS)
  - Values can (and do) exist in both!
    - Copies eliminate WAR hazards
  - Called "value-based" or "copy-based" renaming

- Locations referred to internally by **tags**
  - Register table translates names to tags
    - tag == 0 means "in register file"
    - tag != 0 means "in RS#tag"
  - CDB broadcasts values with tags attached
    - So instructions know what value they are looking at

Tomasulo-Style Register Renaming

- **`ldf f0,X(x1)`**
  - allocate RS#4
  - map **f0** to RS#4
- **`mulf f4,f0,f2`**
  - allocate RS#6
  - **f2** ready, copy *value*
  - **f0** not ready, copy *tag*



- **`addf f2,f6,f5`**
  - key: can write **f2** before mulf executes (why?)

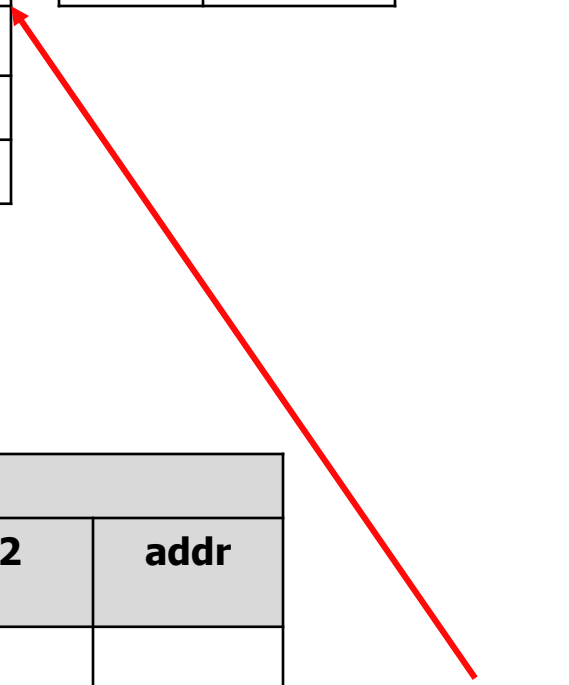
Tomasulo Example: Cycle 1

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1			
mulf f4,f0,f2				
stf f4,Z(x1)				
addi x1,x1,4				
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	RS#2
f2	
f4	
x1	

CDB	
V	T

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	No						
2	load	Yes	ldf		REG[x1]			&X[0]
3	store	No						
4	FP1	No						
5	FP2	No						



Tomasulo Example: Cycle 2

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2		
mulf f4,f0,f2	2			
stf f4,Z(x1)				
addi x1,x1,4				
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	RS#2
f2	
f4	RS#4
x1	

CDB	
V	T

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	No						
2	load	Yes	ldf		REG[x1]			&X[0]
3	store	No						
4	FP1	Yes	mulf		REG[f2]	RS#2		
5	FP2	No						

← Allocate RS, Set reg. status

Tomasulo Example: Cycle 3

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	
mulf f4,f0,f2	2			
stf f4,Z(x1)	3			
addi x1,x1,4				
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	RS#2
f2	
f4	RS#4
x1	

CDB	
V	T

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	No						
2	load	Yes	ldf		REG[x1]			&X[0]
3	store	Yes	stf		REG[x1]	RS#4		&Z[0]
4	FP1	Yes	mulf		REG[f2]	RS#2		
5	FP2	No						

← Allocate RS, no reg. status

Tomasulo Example: Cycle 4

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4		
stf f4,Z(x1)	3			
addi x1,x1,4	4			
ldf f0,X(x1)				
mulf f4,f0,f2				
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	~~RS#2~~
f2	
f4	RS#4
x1	RS#1

CDB	
V	T
[f0]	RS#2

ldf finished
1. write/clear status
2. CDB broadcast

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	Yes	addi	REG[x1]				
2	load	No						
3	store	Yes	stf		REG[x1]	RS#4		&Z[0]
4	FP1	Yes	mulf	CDB.V	REG[f2]	~~RS#2~~		
5	FP2	No						

← Allocate RS, set reg. status

←f0 is ready, Take from CDB

Tomasulo Example: Cycle 5

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5	
stf f4,Z(x1)	3			
addi x1,x1,4	4	5		
ldf f0,X(x1)	5			
mulf f4,f0,f2				
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	RS#2
f2	
f4	RS#4
x1	RS#1

CDB	
V	T

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	Yes	addi	REG[x1]				
2	load	Yes	ldf				RS#1	
3	store	Yes	stf		REG[x1]	RS#4		&Z[0]
4	FP1	Yes	mulf		REG[f2]			
5	FP2	No						

← Allocate RS

Tomasulo Example: Cycle 6

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5+	
stf f4,Z(x1)	3			
addi x1,x1,4	4	5	6	
ldf f0,X(x1)	5			
mulf f4,f0,f2	6			
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	RS#2
f2	
f4	RS#5
x1	RS#1

CDB	
V	T

← No stall on WAW (SB would)

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	Yes	addi	REG[x1]				
2	load	Yes	ldf				RS#1	
3	store	Yes	stf		REG[x1]	RS#4		&Z[0]
4	FP1	Yes	mulf		REG[f2]			
5	FP2	Yes	mulf		REG[f2]	RS#2		

← Allocate RS

Tomasulo Example: Cycle 7

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5+	
stf f4,Z(x1)	3			
addi x1,x1,4	4	5	6	7
ldf f0,X(x1)	5	7		
mulf f4,f0,f2	6			
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	RS#2
f2	
f4	RS#5
x1	~~RS#1~~

CDB	
V	T
[x1]	RS#1

←addi finished
1. write/clear status
2. CDB broadcast

No stall on WAR (SB would)

←Stall DS due to 1 store RS

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	No						
2	load	Yes	ldf		CDB.V		~~RS#1~~	&X[1]
3	store	Yes	stf		REG[x1]	RS#4		&Z[0]
4	FP1	Yes	mulf		REG[f2]			
5	FP2	Yes	mulf		REG[f2]	RS#2		

←x1 (RS#1) ready
Get CDB value

Tomasulo Example: Cycle 8

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5+	8
stf f4,Z(x1)	3	8		
addi x1,x1,4	4	5	6	7
ldf f0,X(x1)	5	7	8	
mulf f4,f0,f2	6			
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	RS#2
f2	
f4	RS#5
x1	

CDB	
V	T
[f4]	RS#4

←mulf finished
1. Don't write/clear status (this tag is for second mulf)
2. CDB broadcast

←Stall DS due to 1 store RS

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	No						
2	load	Yes	ldf					&X[1]
3	store	Yes	stf	CDB.V	REG[x1]	~~RS#4~~		&Z[0]
4	FP1	No						
5	FP2	Yes	mulf		REG[f2]	RS#2		

←f4 (RS#4) ready
Get CDB value
←free

Tomasulo Example: Cycle 9

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5+	8
stf f4,Z(x1)	3	8	9	
addi x1,x1,4	4	5	6	7
ldf f0,X(x1)	5	7	8	9
mulf f4,f0,f2	6	9		
stf f4,Z(x1)				

Reg. Status	
Reg.	T
f0	~~RS#2~~
f2	
f4	RS#5
x1	

CDB	
V	T
[f0]	RS#2

ldf finished
1. write/clear status
2. CDB broadcast

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	No						
2	load	No						
3	store	Yes	stf		REG[x1]			&Z[0]
4	FP1	No						
5	FP2	Yes	mulf	CDB.V	REG[f2]	~~RS#2~~		

←free

←f0 (RS#2) ready
Get CDB value

Tomasulo Example: Cycle 10

Instruction Status (Illustration only)				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5+	8
stf f4,Z(x1)	3	8	9	10
addi x1,x1,4	4	5	6	7
ldf f0,X(x1)	5	7	8	9
mulf f4,f0,f2	6	9	10	
stf f4,Z(x1)	10			

Reg. Status	
Reg.	T
f0	
f2	
f4	RS#5
x1	

CDB	
V	T

stf finished
1. Don't write/clear status
2. CDB broadcast

←DS in same cycle as WB

Reservation Stations and Load/Store Buffers								
T	FU	Busy	op	V1	V2	T1	T2	addr
1	ALU	No						
2	load							
3	store	Yes	stf		REG[x1]	RS#5		&Z[1]
4	FP1	No						
5	FP2	Yes	mulf		REG[f2]			

←free, then allocate

Tomasulo Redux

- RS
  - + Distributed hazard detection
  - + Register renaming eliminates WAW hazards
  - + Copying values to RS eliminates WAR hazards
  - – CDB tag matches require many associative compares

Scoreboard vs. Tomasulo

Scoreboard: Instruction Status				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5+	8
stf f4,Z(x1)	3	8	9	10
addi x1,x1,4	4	5	6	9
ldf f0,X(x1)	5	9	10	
mulf f4,f0,f2	8			
stf f4,Z(x1)	10			

Tomasulo: Instruction Status				
Instruction	DS	IS	EX	WB
ldf f0,X(x1)	1	2	3	4
mulf f4,f0,f2	2	4	5+	8
stf f4,Z(x1)	3	8	9	10
addi x1,x1,4	4	5	6	7
ldf f0,X(x1)	5	7	8	9
mulf f4,f0,f2	6	9	10	
stf f4,Z(x1)	10			

Hazard	Scoreboard	Tomasulo
Insn buffer	Stall in DS	Stall in DS
RAW	Wait in IS	Wait in IS
WAR	Wait in WB	None
WAW	Stall in DS	None

Dynamic Scheduling Summary

- Dynamic scheduling: out-of-order execution
  - Higher utilization, improved performance
- Single F/D reg: structural impediment to OoO execution
  - Instruction buffer: multiple F/D latches
  - Split ID into in-order DS and out-of-order IS
- DS implementations
  - Scoreboard: out-of-order without renaming
  - Tomasulo: out-of-order with renaming
- Next up: **DS + precise state + speculation**

Next up ...

- Dynamic scheduling part 2